

Compiled Semantic Predicates for Approximate Nearest Neighbor Search

Hani M. M. Al-Shater
Independent Research

September 2026

Abstract

Dense retrieval provides an effective similarity primitive, while structured search efficiently handles exact filters. Many useful constraints—such as *minimalist*, *office-appropriate*, or *not sporty*—are latent semantic predicates: they are semantic judgments rather than stored catalog facts, and they are awkward to express as one query–item similarity. We study a *compile once, execute cheaply* design in which expensive semantic supervision is moved offline, items share one low-bit representation, and each latent predicate is compiled into a reusable search-time program.

The original Random Semantic Algebra (RSA) instantiation uses random orthogonal projections, low-bit quantization, and sparse boosted lookup tables. The strongest compact design in our study is instead a centered binary compiler: a 384-dimensional item is stored as 384 sign bits plus two per-item reconstruction scalars (56 bytes/item), while a supervised linear semantic head is quantized to four-bit weights and executed through four packed bit planes (about 216 bytes/predicate). We compare this design against sparse 1/2/4-bit RSA programs, zero-shot MiniLM directions, a full-precision linear semantic proxy, and a PQ64 baseline that compiles the same linear proxy into product-quantization lookup tables.

On 44,072 fashion products, with MiniLM titles as the search-side representation and CLIP images as an independent teacher, Binary1-LS2-int4 reaches 0.337 recall at 20% candidate retention, versus 0.207 for dense truncation, 0.352 for PQ64, and 0.363 for the FP32 semantic proxy. It therefore recovers 83% of the incremental FP32 recall over dense retrieval while using 56 bytes/item and about 216 bytes/predicate. For PQ64 we distinguish persistent state from active execution state: a compact 1.55 KB linear head plus shared codebook can be stored per concept, while the native executor used in our microbenchmark materializes a 65.5 KB lookup table on activation.

We then execute the compiled predicates *inside* HNSW traversal. On the strict first held-out split, using 1,000 queries for each of three predetermined three-predicate sets, $K = 50$, and semantic selectivities from 50% to 2%, live Binary1 execution reaches Traversal Recall@50 between 0.955 and 0.992. A selectivity-aware HNSW over-fetch baseline swept up to $2 \times K/s$ candidates, where s is the eligible fraction, does not come within 0.005 recall of live traversal in any of the 15 predicate-set/selectivity conditions. At 5% and 2% selectivity, live traversal is also faster on average than this largest tested over-fetch point (9.82 vs. 10.27 ms and 14.57 vs. 20.14 ms) while retaining substantially higher traversal recall. The incremental compiled-predicate cost is approximately 96–134 ns per predicate invocation, or 0.16–0.22 of the measured scalar 384-D dot-product cost on the same CPU. The strongest supported result is therefore a systems one: reusable soft predicates can be compiled into tiny programs over a shared low-bit substrate and executed directly during ANN search with a favorable quality–memory–latency trade-off.

1 Introduction

Modern search engines expose two mature computational languages. Dense retrieval expresses semantic similarity through inner products or cosine similarity. Structured retrieval expresses exact predicates—brand, size, category, availability—through inverted indexes, bitmaps, or relational filters. A large class of useful constraints is neither: *minimalist*, *office-appropriate*, *technical*, *quiet luxury*, and explicit negations such as *not sporty* are semantic judgments rather than stored facts.

A dense query vector can absorb these words into one score, but logical composition becomes implicit. At the other extreme, a large VLM or LLM can evaluate each predicate directly, but per-candidate model inference inside

a retrieval loop is expensive. Recent semantic-operator systems and semantic-filter cascades make this execution problem explicit (Patel et al., 2025; Kim et al., 2026; Dong and Wang, 2026).

We study a third design point: *compile once, execute cheaply*. Items are mapped once into a concept-independent low-bit code. Each semantic concept is learned offline as a small executable program over that code. Query understanding selects and composes already-compiled predicates. The same programs can be used as a candidate-side semantic stage, or—in the stronger systems configuration studied here—as soft eligibility predicates evaluated during dense HNSW traversal. Exact catalog filters and downstream ranking remain conventional components.

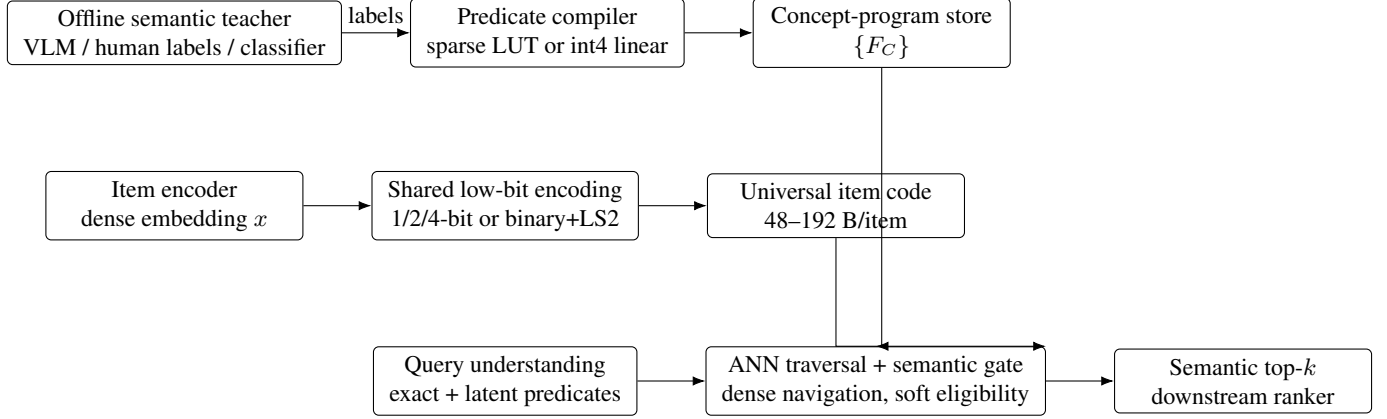


Figure 1: Compiled semantic execution separates expensive supervision from online search. Exact catalog predicates remain conventional filters. Latent predicates are compiled offline into tiny programs over a shared low-bit item representation and can be evaluated as soft eligibility tests during dense ANN traversal.

Our contributions are fourfold. First, we formulate semantic predicate compilation as a two-sided compression problem: both item bytes and predicate-program bytes matter when a catalog serves many reusable concepts. Second, we study two compiler families over a shared search-side embedding: sparse boosted LUT programs on 1/2/4-bit substrates, and a centered 1-bit two-level reconstruction whose int4 semantic heads execute through bitwise population counts. Third, we retain calibrated log-probability composition as a simple query algebra for independently trained positive and negated predicates. Fourth, we integrate the strongest tiny-program compiler into HNSW and evaluate traversal recall, live predicate cost, and latency against a selectivity-aware over-fetch baseline across three independently chosen predicate sets and five semantic selectivities.

The experiments revise an earlier interpretation of the project. Random 4-bit sparse LUTs remain a strong compressed semantic executor, but they are not the best Pareto point in the current benchmark. The centered binary int4 compiler is smaller on both the item and active-program axes and modestly more accurate, while PQ64 remains the strongest compressed quality and large-pool microkernel baseline. We therefore treat random rotation and the particular 4-bit code as design choices, not as the central claim.

We do *not* claim that the binary code replaces ANN geometry, nor that the calibrated product rule is a new probabilistic operator. In fact, the HNSW experiments support the opposite separation: dense similarity should continue to navigate the graph, while compiled semantic programs decide soft result eligibility. The resulting systems object is a programmable semantic filter that can be deployed without materializing one catalog bitmap per learned concept.

2 Problem Setting

Let $x \in \mathbb{R}^D$ be a normalized item embedding and q a query embedding. Dense retrieval scores $q^\top x$. Exact predicates, denoted $E(x)$, are stored catalog facts and should remain conventional filters. A *latent predicate* C is a binary semantic judgment supplied by a teacher $T_C(x) \in \{0, 1\}$, where the teacher may be a VLM, human annotation process, or specialized model.

A semantic compiler chooses a concept-independent item representation $\phi(x)$ and maps labeled examples for concept C into an executable program F_C . If a catalog contains N items and serves K compiled concepts, let B_i denote bytes per item and B_p bytes per persistently stored predicate program. The static memory attributable to semantic execution is

$$M(N, K) = NB_i + KB_p. \quad (1)$$

This distinguishes conventional vector compression, which primarily minimizes B_i , from *joint item-program compression*. A representation with a slightly different item code can become preferable when it makes each additional semantic program much smaller. Some executors may also materialize larger transient state when a predicate is activated; we report persistent and active state separately when that distinction matters.

The target is therefore not only reconstruction fidelity. We seek: (i) no catalog re-encoding when adding a concept; (ii) low B_i ; (iii) low persistent B_p as the semantic vocabulary grows; (iv) fast execution when several predicates are active simultaneously; and (v) composition of independently learned predicates without conjunction-specific training. We evaluate two serving regimes. The candidate-budget experiments measure semantic pruning within a fixed retrieved pool. The systems experiments place the same compiled predicates directly inside HNSW traversal, where dense similarity remains the navigation geometry and semantics

control result eligibility.

3 Low-Bit Semantic Predicate Compilation

3.1 Shared low-bit substrates

The original RSAfamily starts from a normalized search-side embedding $x \in \mathbb{R}^D$, samples a Gaussian matrix, and takes its orthogonal factor $R \in \mathbb{R}^{D \times D}$. For each item,

$$z = xR, \quad q_j = Q_j(z_j) \in \{0, \dots, 2^b - 1\}, \quad (2)$$

where $b \in \{1, 2, 4\}$ and the per-coordinate quantization boundaries are learned from fit data. Orthogonality preserves inner products before quantization. For $D = 384$, packed item storage is 48, 96, and 192 bytes for $b = 1, 2, 4$ respectively. We use random orthogonal rotation for the multi-bit RSAvariants and also test identity versus random rotation for the centered 1-bit sparse model.

3.2 Sparse boosted LUT programs

A concept C can be compiled into a budgeted discrete additive model

$$F_C(q) = b_C + \sum_{j \in J_C} f_{Cj}(q_j) + \sum_{(j,k) \in P_C} g_{Cjk}(q_j, q_k), \quad (3)$$

where each unary f_{Cj} has 2^b entries and each optional pair term g_{Cjk} has 2^{2b} entries. This is a discrete analogue of a generalized additive model with selected interactions (Lou et al., 2013; Nori et al., 2019).

Given current logits $s_i = F_C(q_i)$, define $p_i = \sigma(s_i)$, residual $r_i = y_i - p_i$, and curvature $h_i = p_i(1 - p_i)$. For candidate coordinate j and bin v ,

$$G_{jv} = \sum_{i:q_{ij}=v} r_i, \quad H_{jv} = \sum_{i:q_{ij}=v} h_i, \quad (4)$$

with Newton update $\Delta f_j(v) = G_{jv}/(H_{jv} + \lambda)$ and approximate gain

$$\mathcal{G}_j = \frac{1}{2} \sum_v \frac{G_{jv}^2}{H_{jv} + \lambda}. \quad (5)$$

We greedily add high-gain coordinates, cyclically refit selected unary tables, and then add a small number of pair tables. The main independent-teacher runs use 24 unary terms and two pair terms for RSA1/2/4. Approximate serialized program sizes, including FP32 table values, coordinate identifiers, intercept, and scalar calibration parameters, are 292 bytes for RSA1, 580 bytes for RSA2, and 3,652 bytes for RSA4.

3.3 Centered binary two-level reconstruction

The strongest compact design in the final benchmark is a different compiler. Fit a global centroid μ on search-side embeddings and store the sign pattern of each residual:

$$r = x - \mu, \quad b_j(x) = \mathbb{I}[r_j > 0]. \quad (6)$$

Algorithm 1 Compile a centered-binary semantic predicate

Require: Fit embeddings X , labels y_C , shared centroid μ
 1: fit FP32 linear head (w_C, β_C) on (X, y_C)
 2: quantize w_C to (q_C, a_C, Δ_C) with $q_C \in \{0, \dots, 15\}^D$
 3: pack the four bit planes of q_C
 4: precompute $\sum_j \tilde{w}_{Cj}$ and $\beta_C + \mu^\top \tilde{w}_C$
 5: fit scalar calibration on the held-out calibration split
 6: **return** packed int4 program and calibration parameters

For each item, store two least-squares reconstruction levels,

$$\ell_x = \text{mean}_{j:b_j=0} r_j, \quad h_x = \text{mean}_{j:b_j=1} r_j. \quad (7)$$

The residual coordinate is then approximated by

$$\hat{r}_j = \ell_x(1 - b_j) + h_x b_j. \quad (8)$$

For $D = 384$, the sign vector costs 48 packed bytes and (ℓ_x, h_x) cost two FP32 scalars, for 56 bytes/item.

Train a supervised FP32 linear semantic head

$$s_C(x) = w_C^\top x + \beta_C. \quad (9)$$

We uniformly quantize w_C to unsigned int4:

$$q_j^C = \text{clip}_{[0,15]} \left(\text{round} \frac{w_{Cj} - a_C}{\Delta_C} \right), \quad \tilde{w}_{Cj} = a_C + \Delta_C q_j^C. \quad (10)$$

Under the two-level reconstruction, the approximate predicate score is

$$\hat{s}_C(x) = \beta_C + \mu^\top \tilde{w}_C + \ell_x \left(\sum_j \tilde{w}_{Cj} - P_C(x) \right) + h_x P_C(x), \quad (11)$$

where

$$P_C(x) = \sum_j b_j(x) \tilde{w}_{Cj}. \quad (12)$$

The int4 vector is represented as four packed bit planes M_{Ct} , $t = 0, \dots, 3$. Let B_x denote the 384 packed document bits and $n_+(x)$ their population count. Then

$$P_C(x) = a_C n_+(x) + \Delta_C \sum_{t=0}^3 2^t \text{popcnt}(B_x \wedge M_{Ct}). \quad (13)$$

Thus the 384-dimensional weighted sum reduces to four bitwise AND/popcount passes over six 64-bit words plus a small number of scalar operations. The serialized predicate is approximately 216 bytes: 192 bytes of int4 weights plus quantization and precomputed scalar state. We call this method *Binary1-LS2-int4*; the repository code name is `bbq1_ls2_int4q`. The name reflects inspiration from asymmetric binary quantization, but this is not an exact implementation of Lucene/Elasticsearch BBQ.

3.4 PQ64 compiled linear baseline

Product Quantization (PQ) (Jégou et al., 2011) provides a strong compressed baseline. We partition $D = 384$ into $m = 64$ six-dimensional subspaces and use one 8-bit code per subspace, giving 64 bytes/item. For a learned linear head $w_C = [w_{C1}; \dots; w_{C64}]$ and PQ centroids c_{mk} , precompute

$$L_{Cm}[k] = w_{Cm}^\top c_{mk}, \quad k \in \{0, \dots, 255\}. \quad (14)$$

An item with codes $(q_1, \dots, q_{64})$ is scored by

$$\hat{s}_C^{\text{PQ}}(x) = \beta_C + \sum_{m=1}^{64} L_{Cm}[q_m]. \quad (15)$$

The active scoring kernel used in our native benchmark materializes all 64×256 FP32 LUT entries, about 65,548 bytes including scalar state. That is an execution representation, not a mandatory persistent cost per concept: the system can store the 1,548-byte FP32 linear head (w_C, β_C) plus one PQ codebook shared by all concepts, and construct L_C when the concept is activated. We therefore report PQ64 as 1,548 bytes of persistent per-concept state and 65,548 bytes of active precompiled-LUT state.

3.5 Calibration and semantic algebra

Independently trained concept scores are not on a common evidence scale. On a held-out calibration split, fit a scalar map

$$L_C(x) = a'_C F_C(x) + c_C, \quad p_C(x) = \sigma(L_C(x)). \quad (16)$$

Here F_C may be a sparse LUT score, a binary linear score, a PQ-compiled score, or the FP32 linear score. For positive predicates $\mathcal{Q}^+$ and negated predicates $\mathcal{Q}^-$, we score

$$S_{\mathcal{Q}}(x) = \sum_{C \in \mathcal{Q}^+} \log p_C(x) + \sum_{C \in \mathcal{Q}^-} \log(1 - p_C(x)). \quad (17)$$

This product-of-evidence rule is commutative and associative over the selected predicates. We use it as a simple execution algebra, not as a claim of a novel probabilistic operator.

4 Experimental Setup

Dataset. We use Fashion Product Images Small (Ashraq, 2026). The exploratory mechanism benchmark samples 30k products. The main independent-teacher benchmark uses all 44,072 available products.

Mechanism benchmark. Product titles are encoded by MiniLM (Reimers and Gurevych, 2019; Wang et al., 2020) into 384-dimensional normalized vectors. Fourteen structured fields serve as concept labels. Product titles often contain those labels, so this benchmark is intentionally a representation/compiler mechanism test, not a leakage-free latent-semantic benchmark. Results use the original 65/35 train/test split; binary thresholds are fit on train.

Algorithm 2 Execute a semantic query plan

Require: ANN candidates A , exact filters E , positive concepts $\mathcal{Q}^+$, negative concepts $\mathcal{Q}^-$, budget B

- 1: $A' \leftarrow \{x \in A : E(x) = \text{true}\}$
 - 2: **for** each $x \in A'$ **do**
 - 3: evaluate only the compiled programs requested by the query
 - 4: compute $S_{\mathcal{Q}}(x)$ using Eq. (17)
 - 5: **end for**
 - 6: retain the B highest-scoring candidates
 - 7: **return** survivors to the downstream ranker
-

Independent semantic teacher. For the main benchmark, MiniLM title embeddings provide retrieval and all search-side semantic representations. CLIP ViT-B/32 image semantics (Radford et al., 2021) independently define eight latent concepts: minimalist, office-appropriate, technical/sporty, retro, elegant, relaxed, chunky, and quiet luxury. For each seed, the catalog is split strictly into 52% fit, 13% calibration, and 35% test (22,916 / 5,730 / 15,426 products). Seeds 7, 17, and 27 define three independent splits. For each concept, CLIP scores each image against positive and negative prompt sets; a threshold chosen only on the fit split sets 40% positive prevalence and is frozen for calibration and test. All learned search-side methods see only MiniLM features.

Predicate models. We compare the following representations under the same split and calibration protocol. **RSA1/2/4** use 24 unary boosted LUTs and two pair LUTs per concept; RSA2 and RSA4 use random orthogonal projections with 2-bit and 4-bit quantile bins, while RSA1 is tested with both identity and random-centered sign codes. **Binary1-LS2** uses centered 1-bit documents plus two FP32 per-item reconstruction levels and compiles the same supervised linear semantic head with either FP32 or int4 predicate weights. **PQ64** uses 64 eight-bit PQ subcodes and compiles the FP32 linear head into 64×256 lookup values. **FP32 linear** is a class-balanced logistic model on the original 384-dimensional MiniLM vectors and serves as a search-side semantic ceiling for linear information. We additionally report two training-free MiniLM baselines: a concept-name direction and a positive-minus-negative prompt direction. All supervised predicate scores are scalar-calibrated on the held-out calibration split before composition.

Storage accounting. Reported item bytes assume packed representations. We distinguish persistent per-concept state from larger activation-time state when the executor materializes it. The principal configurations are: FP32 1,536 B/item and 1,548 B/concept; PQ64 64 B/item, a 1,548 B persistent linear head per concept plus one shared 393,216

B codebook, and a 65,548 B materialized LUT per *active* concept in the native executor; Binary1-LS2-int4 56 B/item and 216 B/concept; RSA1 48 B/item and 292 B/concept; RSA2 96 B/item and 580 B/concept; and RSA4 192 B/item and 3,652 B/concept. Joint vocabulary-scale examples use persistent state and exclude graph/container overhead.

Compound-query generation. Queries are generated using fit data only; test labels never influence query selection. Each query contains one to three positive latent predicates, optionally one negated latent predicate, and zero to two exact filters drawn from color, category, article type, gender, and master category. Exact-filter combinations must have sufficient fit support, and the complete semantic conjunction must have at least 100 positive fit examples. We sample 200 valid compound queries per split, yielding 600 query-split cases.

Search protocol. For each query, MiniLM retrieves the top 5,000 held-out test products. Exact constraints are then applied as conventional filters. At each retained fraction $r \in \{1, .5, .4, .3, .2, .1, .05\}$, all methods keep exactly the same number of candidates. The dense baseline orders by query-item cosine similarity. Compiled predicate methods order by Eq. (17); the FP32 proxy uses the same algebra on calibrated full-precision concept logits. An oracle ordering places all teacher-relevant items first and therefore provides a budget-constrained upper bound. Recall and purity are measured against the CLIP-defined semantic conjunction *within the exact-filtered top-5,000 ANN pool*; they are not global catalog recall.

Statistics. We report means over the 600 query-split cases and 95% query-level bootstrap intervals from 2,000 resamples. Paired intervals are computed on per-query differences. Because the three splits share one catalog and one latent vocabulary, these intervals should be read as descriptive uncertainty for this benchmark rather than as 600 fully independent real-world queries.

Native execution benchmark. For the first strict split, the experiment exports the actual held-out FP32 vectors, PQ codes/LUTs, Binary1-LS2 bit codes/corrections/int4 bit planes, and RSA2 codes/programs. A single-threaded Rust release build physically tiles the 15,426 real held-out rows to a 500k-item resident working set and samples candidate sets of 5k, 20k, and 100k items. We evaluate 1, 2, 4, and 8 simultaneously active predicates and report the median of five timed repetitions. The runtime was an x86-64 Colab environment; the CPU model was not recorded, so throughput numbers are implementation evidence rather than hardware-normalized system comparisons. The FP32

path is a scalar reference implementation, not an optimized dense-linear-algebra baseline.

Reviewer-controlled live semantic-HNSW benchmark.

We separately index the first strict held-out split with normalized MiniLM vectors using $M = 24$, construction $ef = 200$, search $ef = 128$, and $K = 50$. We evaluate three pre-determined sets of three active predicates: +minimalist +office-appropriate -technical/sporty; +elegant +quiet-luxury -chunky; and +retro +relaxed -office-appropriate. Each set is evaluated on 1,000 deterministic query items. Thresholds are chosen from the compiled semantic-score distribution to produce target eligibility fractions of 50%, 20%, 10%, 5%, and 2%. Dense similarity alone controls graph navigation; the live method executes the Binary1-LS2-int4 programs to determine result eligibility while invalid nodes remain traversable. Traversal Recall@50 is measured against brute-force dense top- K among items satisfying the *same compiled predicate*; it is an ANN execution metric, not semantic relevance. The controlled materialized traversal uses the same custom graph search with pre-computed semantic scores and isolates live program overhead. A separate ordinary-HNSW baseline retrieves approximately mK/s candidates and post-filters them with the same materialized predicate, where s is actual eligibility and the completed run uses $m \in \{0.75, 1, 1.5, 2\}$. The public harness extends this sweep through $m = 8$ to locate a genuine matched-recall operating point. Exact live/materialized result-ID parity is reported only as a correctness check.

5 Results

5.1 Original 4-bit mechanism benchmark

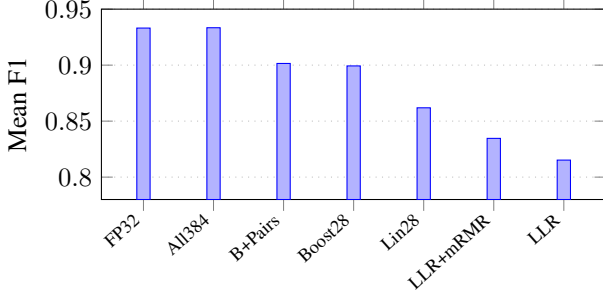
Table 1 and Figure 2 summarize the mechanism benchmark. Compiling all 384 coordinates into 4-bit lookup tables matches the FP32 linear teacher within measurement noise, showing that the 4-bit substrate itself is not the principal bottleneck in this structured-label setting. Sparsifying to 28 unary terms plus four interactions retains 0.9015 mean F1, or 96.6% of the teacher’s F1. The largest gain comes from residual optimization, not interactions: boosted unary LUTs reach 0.8993, whereas independently estimated LLR factors reach 0.8152.

Ablations. At a fixed 192-byte item budget, 384 four-bit coordinates outperform 768 two-bit coordinates (0.8993 vs. 0.8894 mean F1) and 1,536 one-bit random directions (0.8630). Power whitening consistently hurts: $\gamma = 0.25, 0.5, 0.75, 1.0$ produce 0.8944, 0.8654, 0.7993, and 0.6881 F1 respectively. Uniform and quantile 4-bit bins are effectively tied.

These ablations diagnose the original sparse-LUT compiler under structured labels; they should not be read as proving that four bits or random rotation are globally op-

Table 1: Mechanism benchmark (14 concepts).

Method	Mean F1	Mean AP
FP32 linear	0.9331	0.9596
Compiled linear, 384 LUTs	0.9334	0.9596
Boosted LUT + 4 pairs	0.9015	0.9388
Boosted unary LUT	0.8993	0.9372
Compiled linear, 28 coords	0.8619	0.9102
LLR + EB + mRMR	0.8346	0.8902
LLR factor	0.8152	0.8724

**Figure 2:** Residual-optimized LUTs close most of the gap to the full-precision linear classifier in the original structured-label mechanism benchmark.

timal. The independent-teacher benchmark below is the model-selection experiment. There, the full-run sparse ordering is RSA4 > RSA2 > RSA1, while the centered binary linear compiler provides a better compact Pareto point than any sparse RSAvariant.

5.2 Calibration changes the composition regime

Raw addition of independently trained logits performs poorly, especially for three-way conjunctions. After scalar calibration, several soft AND rules converge to similar quality. We use the log-probability product because it is simple, compositional, and needs no conjunction-specific model.

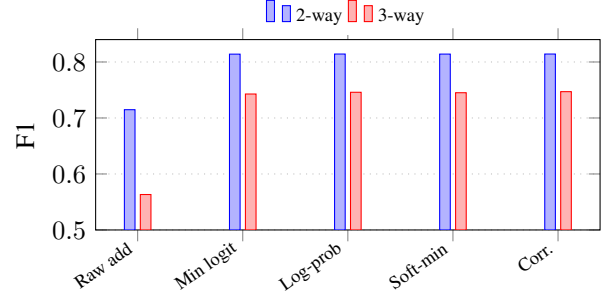
The earlier geometric formulation obtained 0.639 F1 for two-way soft AND and 0.476 for three-way literal boxes. The calibrated program algebra reaches about 0.814 and 0.746–0.747. We therefore retain calibration and the simple product rule in all later supervised methods; the novelty claim is the compiled execution representation, not the probability product itself.

5.3 Cross-modal predicate quality

The main benchmark is deliberately cross-modal: CLIP sees images, whereas every search-side method sees only MiniLM title embeddings. Table 3 reports predicate quality and storage. The FP32 linear head reaches 0.745 mean F1. PQ64 preserves most of this at 0.737. Binary1-LS2 with int4 predicate weights reaches 0.721, ahead of the sparse

Table 2: Composition of independently learned predicates in the mechanism study.

Rule	2-way F1	3-way F1
Raw logit add	0.7148	0.5633
Min calibrated logit	0.8140	0.7428
Log-probability product	0.8142	0.7459
Soft-min ($\tau = 1$)	0.8141	0.7451
Correlation weighted	0.8142	0.7470

**Figure 3:** Calibration, rather than a complex combination rule, drives the improvement in semantic conjunction.

low-bit variants while using the smallest program among the high-performing supervised methods.

Two ablations are especially informative. First, int4 predicate quantization is nearly free: Binary1-LS2 changes from 0.72106 to 0.72053 mean F1 and from 0.77092 to 0.76973 AP when the semantic weight vector is quantized from FP32 to int4. Second, random rotation is not doing much at one bit: centered identity and centered random sparse programs obtain 0.6866 and 0.6875 mean F1 respectively. The final benchmark therefore does not support random rotation as the source of the strongest compact result.

The gap to the image teacher is not purely a quantization effect. Even the uncompressed FP32 MiniLM-side linear proxy is imperfect against image-defined semantics, so the oracle includes information that may simply be absent from product titles.

5.4 Quality–memory Pareto frontier

Table 4 summarizes the main 44,072-product benchmark at 20% candidate retention. Every learned semantic method improves over dense-only truncation, but the compressed methods occupy substantially different memory regimes.

Relative to dense retrieval, Binary1-LS2-int4 improves recall by 0.129 (95% paired bootstrap interval [0.118, 0.140]) and purity by 0.152 [0.141, 0.163]. RSA4 improves recall by 0.123 [0.111, 0.134], while PQ64 improves it by 0.145 [0.132, 0.157]. The binary compiler also exceeds RSA4 by 0.0063 recall [0.0020, 0.0105].

The FP32 linear proxy reaches 0.363 recall. Measured as recovery of the incremental semantic gain over dense

Table 3: Cross-modal predicate quality averaged over three strict splits, with packed storage accounting. For PQ64, the first number is the persistent linear head; the second is the materialized active LUT used by the native executor.

Method	Item B	Program B (stored/active)	Mean F1	Mean AP
FP32 linear	1536	1548	.745	.805
PQ64 compiled linear	64	1548 / 65548	.737	.794
Binary1-LS2, FP32 weights	56	1556	.721	.771
Binary1-LS2, int4 weights	56	216	.721	.770
RSA4 sparse LUT	192	3652	.707	.758
RSA2 sparse LUT	96	580	.702	.747
RSA1 centered random	48	292	.688	.723
RSA1 centered identity	48	292	.687	.724
Zero-shot prompt difference	1536	1536	.604	.565
Zero-shot concept name	1536	1536	.575	.469

Table 4: Main independent-teacher result at 20% retention over 600 query-split cases. Recall and purity are measured within the exact-filtered top-5,000 ANN pool. For PQ64, “stored/active” separates the compact persistent linear head from the materialized lookup table used by the native executor.

Method	Item B	Program B (stored/active)	Recall	Purity
Dense MiniLM	1536	0	.207	.425
Zero-shot prompt difference	1536	1536	.249	.470
RSA1 centered random	48	292	.304	.538
RSA2 sparse LUT	96	580	.321	.561
RSA4 sparse LUT	192	3652	.330	.571
Binary1-LS2-int4	56	216	.337	.578
PQ64 compiled linear	64	1548 / 65548	.352	.594
FP32 linear	1536	1548	.363	.606
Oracle	–	–	.609	.888

retrieval, Binary1-LS2-int4 captures

$$\frac{0.3365 - 0.2075}{0.3626 - 0.2075} = 0.832, \quad (18)$$

PQ64 captures 0.932, and RSA4 captures 0.791. Thus the gap between Binary1-LS2-int4 and PQ64 is 0.0155 absolute recall. The correct storage comparison is subtler than the fully materialized native format suggests. Binary1-LS2-int4 needs about 216 bytes of persistent and active program state. PQ64 can persist the same 1,548-byte FP32 linear head used by the semantic proxy plus one shared PQ codebook, then materialize the 64×256 float lookup table only when the predicate is activated. The 65,548-byte number is therefore an *active precompiled-LUT representation*, not an intrinsic persistent cost per PQ concept.

Table 5 makes the two-sided compression objective concrete without treating PQ lookup materialization as mandatory persistent state. Binary1-LS2-int4 remains the smallest joint payload in this illustrative regime. PQ64 is only eight bytes/item larger and its compact persistent head is about $7.2\times$ larger than the binary program; its 65.5 KB lookup table is an activation-time representation. The present native PQ benchmark uses those pre-materialized LUTs and therefore does not measure their construction cost. The

Table 5: Illustrative persistent semantic payload for 5M items and 100k compiled concepts. Decimal MB; excludes HNSW graph edges and container overhead. PQ64 includes 100k compact linear heads plus one shared float codebook, not one persistent 65.5 KB LUT per concept.

Method	Items MB	Programs/store MB	Total MB
Binary1-LS2-int4	280.0	21.6	301.6
PQ64 linear heads	320.0	155.2	475.2
RSA2 sparse LUT	480.0	58.0	538.0
FP32 linear	7680.0	154.8	7834.8

table is representation arithmetic, not a claim about total resident-set size in a production service.

5.5 Composition depth is confounded by prevalence

A raw stratification by the number of latent predicates suggests larger semantic gains for deeper conjunctions. However, deeper conjunctions also have lower teacher prevalence inside the candidate pool, which makes fixed-budget pruning easier. We therefore re-analyzed the per-query 20% recall deltas with pool prevalence as a covariate. In an exploratory OLS regression of method-minus-dense recall

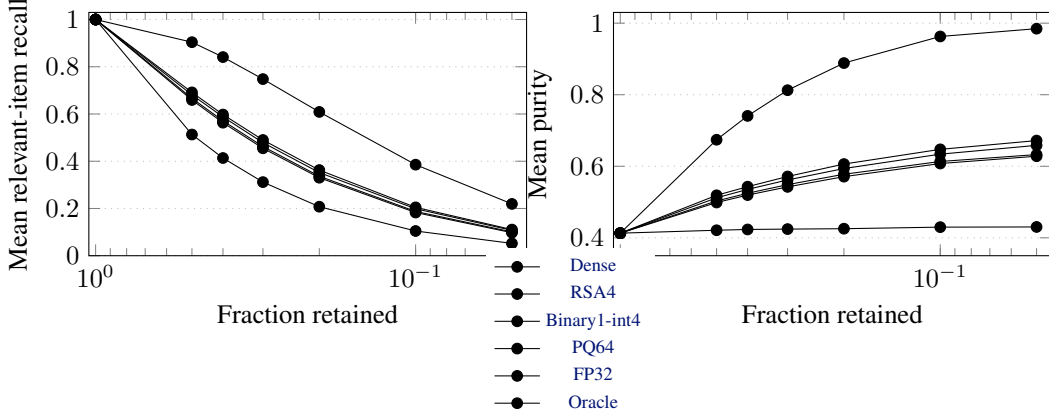


Figure 4: Candidate-budget frontier on the independent-teacher benchmark. Binary1-LS2-int4 is the strongest tiny-program point; PQ64 is the strongest compressed quality point; FP32 remains the best search-side linear proxy.

Table 6: Single-threaded native throughput at 100k candidates (million candidates/s). Real held-out rows/programs, 500k resident items, median of five repetitions. The FP32 column is a scalar reference implementation rather than an optimized BLAS baseline.

Predicates	FP32 scalar	PQ64	Binary1-int4	RSA2
1	1.301	7.010	5.241	6.778
2	1.042	5.812	4.534	4.760
4	0.747	3.802	3.200	3.004
8	0.413	2.044	1.977	1.737

on latent count and teacher prevalence, the latent-count coefficient is -0.0033 for Binary1-LS2-int4 ($p = 0.58$) and -0.0026 for RSA4 ($p = 0.67$). We therefore find no evidence in this benchmark that composition depth itself increases the advantage after accounting for prevalence, and we do not use the raw depth trend as a headline result.

5.6 Native execution on real codes and programs

The quality experiment exports the actual first-split held-out representations and learned programs to a Rust executor. Table 6 reports throughput at a 100k-candidate pool with a 500k-item resident working set. PQ64 is fastest at this large pool across all predicate counts. Binary1-LS2-int4 closes the gap as more predicates are fused: with eight active predicates it reaches 1.98M candidates/s, within 3.3% of PQ64’s 2.04M/s.

These measurements isolate the resident scoring kernels rather than end-to-end search. PQ64 combines the best compressed quality with the best large-pool throughput in this implementation, while Binary1-LS2-int4 remains close at higher predicate counts and uses a much smaller persistent program. We do not interpret the ratio to the scalar FP32 loop as a general speedup over optimized dense linear algebra.

5.7 Live semantic predicates inside HNSW

The preceding microkernel asks how quickly a fixed candidate array can be scored. We next move the compiled predicate into ANN traversal itself. Dense similarity remains the HNSW navigation score; semantic programs only decide whether a visited node may enter the valid result beam. Semantically invalid nodes remain traversable, so the soft predicate does not disconnect the graph. A dense bound is checked before semantic execution, which avoids touching the predicate program for neighbors that cannot improve the current valid beam.

The reviewer-controlled benchmark uses the strict first held-out split of the 44,072-product source catalog, 1,000 deterministic query items, $K = 50$, $ef = 128$, and three predetermined sets of three active semantic predicates. For each predicate set, thresholds induce target eligibility fractions of 50%, 20%, 10%, 5%, and 2%. Traversal recall is measured against brute-force dense top- K among items passing the *same compiled predicate*; it is therefore a search-execution metric rather than end-to-end semantic relevance. The comparison baseline performs ordinary HNSW search for approximately mK/s candidates and materializes the same semantic predicate only after retrieval, where s is the actual eligible fraction and $m \in \{0.75, 1, 1.5, 2\}$ in the completed run.

The result is more informative than the earlier fixed-postfilter comparison. At 50–10% selectivity, a $2\times$ over-fetch budget is somewhat faster than live traversal, but it gives up 0.19–0.27 absolute Traversal Recall@50. At 5% and 2% selectivity, live traversal is both faster and substantially higher recall than that same budget. Most importantly, *no* tested over-fetch point comes within the predeclared 0.005 recall tolerance of live traversal in any of the 15 conditions. We therefore do not report a matched-recall latency ratio from this run; the correct statement is that the

Table 7: Reviewer-controlled live semantic HNSW result. Values are means over three predetermined predicate sets, each evaluated on 1,000 queries. “2× OF” is the largest completed selectivity-aware over-fetch budget, approximately 2K/s. None of the 15 predicate-set/selectivity conditions reaches live recall within 0.005 at any tested over-fetch point.

Eligible	Live trav. recall	2× OF recall	Recall gap	Live ms	2× OF ms	OF/live	Pred. ns
50%	.9833	.7923	-.1910	2.23	1.61	.73	102
20%	.9826	.7098	-.2728	4.58	3.46	.76	118
10%	.9794	.7180	-.2614	6.79	5.95	.88	124
5%	.9752	.7482	-.2270	9.82	10.27	1.05	130
2%	.9778	.8420	-.1357	14.57	20.14	1.38	132

completed over-fetch range failed to reach the live-recall operating point.

The live method reaches between 0.955 and 0.992 Traversal Recall@50 across the individual conditions. Exact result-ID parity with the corresponding custom materialized traversal is 1.0 and is treated only as a correctness check. The incremental live-minus-materialized custom-traversal cost is approximately 96–134 ns per predicate invocation. On the same Intel Xeon 2.20 GHz CPU, the scalar 384-D normalized-dot kernel costs 617 ns, so the incremental predicate cost is about 0.16–0.22 of one dense distance computation. This ratio is stable enough across predicate sets and selectivities to support the narrower claim that predicate execution itself is not the dominant cost of the traversal.

An extended sweep to larger over-fetch budgets is implemented in the public reviewer harness so that a true matched-recall point can be found rather than inferred. Until that sweep is run, the paper makes no claim that live traversal is faster than every possible post-filter over-fetch configuration. The checked-in summary used for Table 7 is `paper/icml/data/semantic_hnsw_reviewer_2x_summary.csv`.

6 Systems Interface and Latency Scope

6.1 Bring-your-own retrieval and embeddings

The intended deployment boundary remains narrow: a host search engine owns document lifecycle, exact filters, lexical/vector indexing, and the downstream ranker. Given any precomputed item embedding matrix $X \in \mathbb{R}^{N \times D}$, the semantic layer fits one catalog-wide binary encoder, writes packed item codes once, and separately stores compiled predicate programs. Changing the embedding model or the catalog-wide binary transform creates a new index generation and requires predicates to be recompiled; adding a new semantic concept writes only a new program and does not rewrite item codes.

There are two serving modes. In the candidate-side mode, an existing ANN engine passes row IDs to `SemanticExecutor`, which scores and prunes them. In the integrated mode, the same compiled programs are called from HNSW traversal: dense similarity remains the navigation geometry, while the learned predicate decides whether

a visited point may enter the valid result beam. Invalid points remain traversable. The integrated experiment is the stronger systems result because it avoids materializing a large candidate pool before semantic filtering.

The repository exposes the portable representation through `BinarySemanticIndex` and `ProgramStore`. The item payload is a packed sign-bit file plus two f32 correction values per item. A Binary1-LS2-int4 predicate contains four packed int4 weight bit planes plus scalar scoring/calibration metadata. For $D = 384$, the accounting used throughout the paper is 56 bytes/item and about 216 bytes/predicate. The Python reference loader memory-maps the item payload. The current Rust prototypes read the resident files into memory; mmap/segment management remains deployment work rather than a claimed result.

6.2 Joint item–program memory

The persistent memory objective is Eq. 1. This differs from vector compression alone because a semantic search system may keep a large reusable concept vocabulary while activating only a small subset for each query. The comparison must distinguish *persistent* from *active* state. Binary1-LS2-int4 uses essentially the same 216-byte representation for both. PQ64 can persist a 1,548-byte linear head per concept plus one shared codebook and materialize a 65,548-byte lookup table only when that concept is activated. Table 5 therefore uses the compact persistent PQ representation rather than charging one permanent LUT per concept.

This correction changes the scale of the memory claim. Binary1-LS2-int4 still has the smallest joint payload in the illustrative 5M-item/100k-concept calculation, but the relevant persistent-program ratio to the PQ head is about $7.2\times$, not $303\times$. The larger 65.5 KB PQ number remains relevant to active cache footprint and to the native microkernel, which consumes pre-materialized LUTs, but it is not an intrinsic storage requirement.

6.3 Latency accounting

End-to-end search latency should be decomposed rather than attributing the entire query path to the semantic kernel:

$$T_{\text{search}} = T_{\text{ANN+semantic}} + T_{\text{exact}} + T_{\text{topk}} + T_{\text{rank}} + T_{\text{RPC}}. \quad (19)$$

For candidate-side deployment, $T_{\text{ANN}+\text{semantic}}$ can be decomposed further into ANN traversal plus a semantic scan. For integrated deployment, ANN traversal and soft-predicate execution are deliberately interleaved, so reporting a single measured search-stage latency is more faithful.

The older native results in Table 6 measure a single-threaded semantic *microkernel*: real learned item codes and programs are resident, but ANN traversal and graph book-keeping are excluded. Table 8 reports those measurements as elapsed time, which isolates raw scoring cost.

At a 5k candidate pool, Binary1-LS2-int4 ranges from 0.48 ms for one predicate to 1.99 ms for eight predicates in that historical run. The CPU model was not recorded, so the absolute numbers are retained only as microkernel scaling evidence. We do not use the ratio to the scalar FP32 loop as a headline speedup. The newer HNSW experiment in Table 7 is the more relevant search-stage result because it includes graph traversal, valid-beam maintenance, and live compiled predicate execution.

6.4 Live predicate execution during traversal

The integrated Rust path uses a dense-first rule. A discovered neighbor is first scored by dense similarity. If it cannot improve the current valid *ef* beam, it is discarded before touching semantic state. Otherwise its Binary1-LS2-int4 predicate program is evaluated and cached once for that query. Valid points may enter the result beam; invalid points are still inserted into the dense navigation frontier, preserving graph connectivity. Semantic scores never modify navigation priority in the final design.

This separation emerged empirically. Directly steering the HNSW priority by a weighted semantic score reduced visited nodes but damaged Traversal Recall@50. Explicit two-hop bridge expansion added many candidate edges without improving recall. The final algorithm therefore keeps geometry and eligibility separate:

navigate by $s_{\text{dense}}(q, x)$, admit result if $S_{\text{semantic}}(x) \geq \tau$.
(20)

In the 1,000-query reviewer benchmark, three independently chosen three-predicate sets are tested at 50%, 20%, 10%, 5%, and 2% eligibility. Live traversal reaches 0.955–0.992 Traversal Recall@50 across the 15 conditions. Relative to the same custom traversal with semantics materialized for free, the incremental program cost is approximately 96–134 ns per predicate invocation. The measured scalar 384-D normalized-dot kernel on the same Intel Xeon 2.20 GHz CPU costs 617 ns, putting the predicate overhead at roughly 0.16–0.22 dense-distance equivalents per invocation.

The selectivity-aware over-fetch baseline is deliberately separated from the older `hnsw.rs` filtered-search comparison. For an eligible fraction s , the completed reviewer run asks ordinary HNSW for approximately mK/s candidates

with $m \in \{0.75, 1, 1.5, 2\}$, then applies the same materialized semantic predicate. No tested point reaches live traversal recall within 0.005 in any of the 15 conditions. We therefore do not attribute the older custom-vs-library latency gap to the compiled method and do not claim a universal matched-recall speedup from the current completed sweep. At the largest completed budget, live traversal becomes both faster and higher recall at 5% and 2% selectivity; at higher selectivities over-fetch is faster but materially lower recall.

Exact result-ID parity between live execution and the corresponding custom materialized traversal is 1.0 across the completed sweep. This is a correctness assertion: it verifies that executing the Binary1 programs online reproduces the same custom traversal decision as precomputing their semantic scores. It is not itself a search-quality result.

6.5 Composition-aware early termination

The calibrated composition rule also provides a safe upper bound when several predicates are evaluated sequentially. Each positive term is $\log \sigma(L_C) \leq 0$ and each negated term is $\log \sigma(-L_C) \leq 0$. After evaluating any prefix P of a query plan,

$$S_P(x) \geq S_Q(x), \quad (21)$$

so the partial score is an upper bound on the final score. If a top- k heap already has threshold τ and $S_P(x) \leq \tau$, the item cannot recover after evaluating remaining predicates. The current live-HNSW gate evaluates the active predicate set exactly, but this bound remains available for future multi-predicate short-circuiting and for candidate-side top- k composition.

6.6 Reproducibility and remaining systems work

The reviewer benchmark is reproducible through `experiments/semantic_hnsw_reviewer_sweep.py` and the public Colab `notebooks/rsa_semantic_hnsw_reviewer_colab.ipynb`. The optimized harness exports only the assets needed by HNSW, verifies unit-normalized retrieval vectors, derives selectivity thresholds from the compiled programs, builds the graph once per predicate set, computes brute-force dense scores once per query, and sweeps all gates and over-fetch budgets in that same process. It writes raw, summary, matched-frontier, predicate-cost, and environment artifacts including the repository commit, CPU model, Python version, and Rust version. The completed 2× summary used in the paper is `paper/icml/data/semantic_hnsw_reviewer_2x_summary.csv`.

The public harness now extends the over-fetch sweep to $m \in \{0.75, 1, 1.5, 2, 3, 4, 6, 8\}$ specifically to search for a genuine matched-recall point rather than stopping at a budget that fails to match. The remaining systems questions are: (i) the final matched-recall frontier from that extended sweep; (ii) mmap/zero-copy loading and direct integration with a production ANN index rather than an

Table 8: Single-thread semantic microkernel latency in milliseconds (median of five repetitions, real held-out rows/programs tiled to 500k resident items). These are not end-to-end search latencies. The FP32 path is a scalar reference implementation rather than optimized BLAS.

Candidates	Predicates	FP32 scalar	PQ64	Binary1-LS2-int4
5k	1	3.049	0.540	0.484
5k	4	6.439	0.971	1.130
5k	8	11.812	2.108	1.993
20k	1	15.530	2.218	2.190
20k	4	27.236	4.120	5.694
20k	8	48.241	8.910	8.617
100k	1	76.846	14.265	19.080
100k	4	133.781	26.302	31.248
100k	8	242.037	48.923	50.583

extracted research graph; (iii) p50/p95/p99 under concurrency and thread scaling; (iv) update/tombstone and index-generation semantics; (v) stores of 10^2 – 10^6 concepts with realistic program lookup/cache behavior; and (vi) end-to-end relevance on natural search or recommendation traffic. The current result establishes that live learned soft predicates are cheap enough to execute during ANN traversal; it does not establish production p99 or distributed-service latency.

7 Related Work

Quantized retrieval. Product Quantization compresses vectors for ANN search (Jégou et al., 2011); random-hyperplane hashing connects sign agreement to angle (Charikar, 2002). RaBitQ provides randomized low-bit distance estimation with error bounds (Gao and Long, 2024). MUVIRA compiles multi-vector similarity into fixed-dimensional encodings for MIPS (Dhulipala et al., 2024). Our setting differs in the object being compressed: a catalog may serve many reusable semantic functions, so predicate-program storage and multi-predicate execution matter in addition to item-code size. The PQ64 compiled-linear baseline is consequently important rather than ancillary.

Filtered vector search. Filtered ANN combines vector similarity with structured predicates and introduces non-trivial recall/latency trade-offs depending on selectivity and correlation (Chronis et al., 2025). Our final HNSW experiment moves learned soft predicates into this systems setting. The distinction is operational: a structured filter is normally materialized catalog state, whereas our eligibility test is computed from a 56-byte shared item code and a 216-byte independently deployable learned program. Dense similarity continues to navigate the graph.

Semantic operators and proxy execution. LOTUS formalizes natural-language semantic operators and their op-

timization (Patel et al., 2025). Recent semantic-filter work studies learned proxies and adaptive cascades to reduce expensive oracle calls (Kim et al., 2026). Compilation-based semantic execution similarly moves semantic interpretation out of per-item runtime (Dong and Wang, 2026). The design studied here occupies a lower-level systems point: one shared low-bit item substrate, independently compiled semantic predicates, calibrated composition, and direct execution inside a retrieval loop.

8 What the Experiments Support

The experiments narrow the claim substantially relative to the initial RSA hypothesis. First, random orthogonal rotation is not required for the strongest compact result: centered 1-bit identity codes support the best tiny-program design, and the 1-bit sparse identity/random ablation is effectively tied. Second, four-bit sparse LUTs are useful but not Pareto-optimal in the final benchmark. Third, the calibrated log-probability product is operationally useful but should be viewed as a simple composition rule rather than a new algebraic theorem.

The more distinctive result is *joint item-program compression plus live search execution*. Binary1-LS2-int4 uses 56 bytes/item and about 216 bytes of persistent program state while recovering 83% of the FP32 semantic gain over dense truncation. PQ64 is the stronger compressed quality point, but its memory trade-off must be stated fairly: the 65.5 KB lookup table used by the native executor can be materialized only for active predicates, while persistent storage can remain a 1.55 KB linear head per concept plus a shared codebook. Under that accounting, the illustrative 5M-item/100k-concept payload is about 302 MB for Binary1-LS2-int4 versus about 475 MB for PQ64, not multiple gigabytes of mandatory PQ program state.

The reviewer-controlled HNSW result adds a second axis. Using 1,000 queries for each of three predetermined three-predicate sets, live execution reaches 0.955–0.992 Traversal Recall@50 from 50% down to 2% eligibility. A selectivity-

aware over-fetch sweep through $2 \times K/s$ does not match live recall within 0.005 in any of the 15 conditions. At 50–10% eligibility the largest completed over-fetch point is faster but substantially lower recall; at 5% and 2% live traversal is both faster and higher recall. The incremental semantic-program cost is approximately 96–134 ns per predicate invocation, or about 0.16–0.22 of the scalar 384-D dot-product cost measured on the same CPU.

This does *not* mean that semantic scores should steer ANN geometry. Our direct semantic-priority experiment reduced traversal work but damaged recall, while explicit two-hop bridge expansion added work without a meaningful quality gain. The supported algorithm is simpler: dense navigation, soft semantic eligibility, invalid nodes still traversable. It also does not yet establish a universal matched-recall speedup over post-filtering: the completed over-fetch range failed to reach the live operating point, and the public harness now extends the sweep to larger budgets specifically to locate that frontier.

9 Limitations and Next Experiments

The main semantic-quality benchmark still covers one fashion catalog, one retrieval encoder, and eight CLIP-defined latent concepts. Teacher labels come from prompt-based CLIP image comparisons rather than human relevance judgments, and concept prevalence is standardized to 40% on each fit split. The 200 queries per split are synthetically generated from a fixed latent vocabulary and fit-supported exact filters; they are useful for controlled evaluation but are not a natural search-log distribution.

The candidate-budget recall numbers remain conditional on the exact-filtered top-5,000 MiniLM pool. They measure how well a cheap semantic executor allocates a downstream budget, not end-to-end catalog recall. The HNSW experiment addresses execution inside ANN traversal, but it still uses the first strict held-out split and synthetic selectivity thresholds over three active predicates per query. Traversal recall is defined relative to the same compiled predicate, so it evaluates ANN execution fidelity rather than semantic truth.

The Binary1-LS2-int4 compiler is *BBQ-inspired*, not an exact Lucene/Elasticsearch BBQ implementation, so the paper should not be read as a direct benchmark against that production kernel. Likewise, the older `hnsw_rs` filtered-search timing is retained only as library context; the paper no longer attributes the custom-vs-library latency gap to the compiled predicate mechanism. The controlled comparison that matters is live traversal versus selectivity-aware over-fetch on the same graph and the live-versus-materialized custom traversal used to isolate incremental predicate cost.

The Rust implementation is single-threaded and resident-memory. It does not yet measure p99 under concurrency, mmap/segment behavior, distributed RPC, updates/tombstones, or a production graph layout. The

100k-concept memory table is representation arithmetic; the current quality benchmark learns only eight concepts, so large-vocabulary cache behavior remains unmeasured. The current graph contains only the strict held-out split rather than a multi-million-item production-scale catalog.

The highest-value next experiments are therefore: completing the extended matched-recall over-fetch sweep; scaling the resident graph to millions of items and reporting QPS/core; a second dataset/domain; human or stronger held-out semantic judgments; natural LLM-generated search plans; and stores of 10^2 – 10^6 concepts. A representation-learning direction is to make semantic predicates more locally linear—for example, cluster-conditional linear heads or a small geometry-preserving transform trained for semantic separability—and then test how much of the resulting teacher improvement survives compilation into the same low-bit substrate.

10 Conclusion

The study began with a random 4-bit semantic substrate and sparse LUT programs, but the final experiments point to a broader conclusion. Semantic supervision can be compiled offline into reusable search-time predicates over a shared low-bit item representation, and both sides of that interface—item code and predicate program—matter. On the 44,072-product independent-teacher benchmark, Binary1-LS2-int4 reaches 0.337 recall at 20% retention versus 0.207 for dense truncation, 0.352 for PQ64, and 0.363 for the FP32 linear proxy. It uses 56 bytes/item and about 216 bytes/concept and recovers 83% of the FP32 incremental recall over dense retrieval.

The systems result goes beyond candidate reranking. Across three predetermined predicate sets and 1,000 queries per set, live compiled predicates maintain approximately 0.96–0.99 Traversal Recall@50 from 50% down to 2% eligibility. No selectivity-aware over-fetch point up to $2 \times K/s$ reaches that recall within 0.005 in the completed sweep, and the compiled predicate adds only about 96–134 ns per invocation. Exact live/materialized ID parity is used only as a correctness check.

The strongest supported thesis is therefore not that randomness, four-bit quantization, or a particular conjunction operator is uniquely novel. It is that *compiled semantic predicates can become first-class search artifacts*: tiny learned programs over one shared item substrate, selected by query understanding, composed when needed, and executed cheaply enough to participate directly in ANN traversal.

Impact Statement

If successful at scale, compiled semantic predicates could reduce online model inference and energy use in retrieval systems while enabling richer natural-language constraints. The same mechanism could also encode biased or poorly

specified teacher judgments at scale; deployed systems should therefore audit predicate labels, calibration, subgroup behavior, and failure modes, and should preserve conventional safety and policy filters outside the learned semantic layer.

References

- H. Jégou, M. Douze, and C. Schmid. Product Quantization for Nearest Neighbor Search. *IEEE TPAMI*, 33(1):117–128, 2011.
- M. S. Charikar. Similarity Estimation Techniques from Rounding Algorithms. In *STOC*, pp. 380–388, 2002.
- J. Gao and C. Long. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *arXiv:2405.12497*, 2024.
- L. Dhulipala, M. Hadian, R. Jayaram, J. Lee, and V. Mirrokni. MUVERA: Multi-Vector Retrieval via Fixed Dimensional Encodings. In *NeurIPS*, 2024.
- Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki. Filtered Vector Search: State-of-the-Art and Research Opportunities. *PVLDB*, 18(12):5488–5492, 2025.
- L. Patel, S. Jha, M. Pan, H. Gupta, P. Asawa, C. Guestrin, and M. Zaharia. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *PVLDB*, 18(11):4171–4184, 2025.
- K. Kim, M. Catheland, and A. Ailamaki. Fast LLM-Based Semantic Filtering: From a Unified Framework to an Adaptive Two-Phase Method. *arXiv:2606.08090*, 2026.
- W. Dong and Y. Wang. From Interpretation to Compilation: Compilation-Based Execution of Semantic Operators [Vision]. *arXiv:2607.13407*, 2026.
- Y. Lou, R. Caruana, J. Gehrke, and G. Hooker. Accurate Intelligible Models with Pairwise Interactions. In *KDD*, pp. 623–631, 2013.
- H. Nori, S. Jenkins, P. Koch, and R. Caruana. InterpretML: A Unified Framework for Machine Learning Interpretability. *arXiv:1909.09223*, 2019.
- N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *EMNLP-IJCNLP*, pp. 3982–3992, 2019.
- W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *NeurIPS*, 2020.
- A. Radford et al. Learning Transferable Visual Models From Natural Language Supervision. In *ICML*, pp. 8748–8763, 2021.
- Ashraq. Fashion Product Images Small. Hugging Face Datasets, accessed 2026.

A Additional Experimental Detail

A.1 Original geometry sanity check

The original mechanism run measured correlation 0.9102 between source cosine similarity and 1-bit Hamming agreement, and 0.9761 between source cosine and a 4-bit negative- ℓ_1 surrogate over random held-out pairs. These values motivated the first low-bit substrate experiments but are not ANN recall guarantees and are not evidence that random rotation is necessary.

A.2 Storage accounting

For $D = 384$, source FP32 embeddings require 1,536 bytes/item. The final benchmark uses the following packed accounting:

Method	Item B	Stored program B	Active program B
FP32 linear	1536	1548	1548
PQ64 compiled linear	64	1548	65548
Binary1-LS2-int4	56	216	216
Binary1-LS2-FP32	56	1556	1556
RSA1 sparse	48	292	292
RSA2 sparse	96	580	580
RSA4 sparse	192	3652	3652

Binary1 stores 48 bytes of sign bits plus two FP32 reconstruction levels. Its int4 predicate stores 192 bytes of packed weights plus quantization/precomputed scalar state. PQ64 stores 64 one-byte subcodes per item. The native PQ executor uses 64×256 FP32 table values plus scalar state, giving 65,548 bytes of active LUT state per predicate. That table need not be persisted for every concept: the underlying 384-dimensional FP32 linear head is 1,548 bytes and can be combined with one shared PQ codebook to materialize a LUT on activation. Sparse RSA program sizes count FP32 LUT values, unsigned 16-bit coordinate indices, intercept, and scalar calibration parameters. These numbers intentionally exclude generic container and allocator overhead.

For PQ64 with $m = 64$, 8-bit subcodes, and $D = 384$, the shared float codebook contains $64 \times 256 \times 6$ values, or 393,216 bytes. Thus the illustrative 5M-item/100k-concept persistent accounting in Table 5 charges 320 MB for item codes, 154.8 MB for the 100k linear heads, and about 0.4 MB for the shared codebook. It does not charge 65.5 KB times 100k as permanent state.

A.3 Binary execution identity

For the int4 binary program, the native kernel uses six 64-bit words per item and four predicate bit planes. The weighted positive-side sum is obtained from four AND/popcount reductions as in Eq. (13); the two per-item reconstruction levels then recover the approximate linear score through Eq. (11). This is the kernel benchmarked by the Rust actual executor.

A.4 Reproducibility

The final quality comparison is produced by `experiments/binary_bbq_predicates.py` with configs/`binary_bbq.yaml`. The reported result bundle is `20260904T125016Z_binary_3ad715855d`, generated at Git commit `2b0f5ed`. It contains all 44,072 products, seeds 7/17/27, a top-5,000 retrieval pool, 200 fit-generated compound queries per seed, and the full per-query, predicate, paired-delta, and Pareto outputs. The quality run reports an NVIDIA A100-SXM4-40GB for embedding/teacher computation.

The same run exports first-seed real held-out representations and learned programs. `experiments/export_native_finalists.py` converts them to raw binary assets, and `rust/semantic_engine/src/bin/actual.rs` benchmarks FP32, PQ64, Binary1-LS2-int4, and RSA2 on a single CPU thread. The source test set has 15,426 items; rows are physically repeated only to construct the 500k-item resident working set. Candidate sets contain 5k, 20k, or 100k sampled resident indices, predicate counts are 1/2/4/8, and reported timings are medians of five repetitions. The public Colab launcher is `notebooks/rsa_finalists_full_native_colab.ipynb`. The FP32 native path is a scalar reference loop and is not used to claim a general speedup over optimized dense linear algebra.

The reviewer-controlled HNSW experiment is produced by `experiments/semantic_hnsw_reviewer_sweep.py`. The focused exporter `experiments/export_hnsw_assets.py` writes only the normalized held-out retrieval vectors, Binary1-LS2 item sidecar, and eight calibrated semantic programs needed by the graph benchmark. The Rust executable `rust/semantic_engine/src/bin/semantic_hnsw_reviewer.rs` builds one HNSW graph per predicate set, computes brute-force dense scores once per query, and sweeps all five selectivity gates and all requested over-fetch budgets inside that process. The completed result uses three predetermined three-predicate sets, 1,000 queries each, $K = 50$, $ef = 128$, $M = 24$, and $efConstruction = 200$. Traversal recall is defined as recall of brute-force dense top- K among items passing the same compiled semantic predicate. The public Colab is `notebooks/rsa_semantic_hnsw_reviewer_colab.ipynb`.